

{Hibernate Upgrade Probleme}

UNTERTITEL

Aug 14, 2026

erhard.siegl@gepardec.com

simon.gartner@gepardec.com

Inhaltsverzeichnis

Inhaltsverzeichnis.....	1
Über diese Dokument.....	2
Hibernate in JBoss EAP.....	2
EAP 7.....	2
EAP 8.....	2
Release Notes.....	3
Problemsammlung.....	3
Maven Koordinaten.....	3
Genauere Checks beim Deployment umgehen.....	3
Case sensitive bei Native Queries.....	3
Funktionsaufrufe bei Named Queries.....	3

Queries ohne Select.....	4
Keine automatischen Casts z.B. bei Vergleichen.....	4
Integer.....	4
Datum.....	4
Updateable und Insertable.....	4
Anpassung bei @FilterDef und @ParamDef.....	5
Evolution der UserTypes.....	5
Hibernate Validator und EL-Abhängigkeiten.....	5
Explizite Schema-Trennung in @Table.....	5
Rückgabetypen bei Native Queries.....	6
Integer Overflow bei NUMBER(10).....	6
Bulk-Operationen und HTE-Tabellen.....	6
Vergleich mit NULL in HQL / JPQL.....	6
Strengere Detached Prüfung.....	7
Cast as date in @Filter.....	7
Hibernate Java Time Handling.....	7

Über diese Dokument

Internes Dokument zur Sammlung von Problemen mit Hibernate beim Versionsupgrade und möglichen Lösungen.

Hibernate in JBoss EAP

EAP 7

Component	7.4	7.3	7.2	7.1	7.0
Hibernate Core	5.3.20.Final	5.3.20.SP1	5.3.17.Final	5.1.17.Final	5.0.16.Final

EAP 8

Component	8.1	8.0
-----------	-----	-----

Release Notes

- [Hibernate 6.0 Final](#)
- [Migration Guide](#)

Problemsammlung

Maven Koordinaten

Die GroupId für Hibernate-Artefakte wurde von *org.hibernate* auf *org.hibernate.orm* umgestellt. Diese Änderung muss in der Projektkonfiguration nachgezogen werden.

Zur Automatisierung bietet sich folgendes OpenRewrite-Rezept an:

```
- org.openrewrite.maven.ChangeManagedDependencyGroupIdAndArtifactId
  oldGroupId: org.hibernate
  oldArtifactId: hibernate-core
  newGroupId: org.hibernate.orm
  newArtifactId: hibernate-core
```

Genauere Checks beim Deployment umgehen

Umgehungslösung: In persistence.xml folgende Property setzen:

hibernate.query.jpql_strict_compliance=false

Diese Lösung ist gefährlich.

Case sensitive bei Native Queries

Funktionsaufrufe bei Named Queries

(Oracle- oder Custom-)Funktionen die in JPA nicht definiert sind können nicht mehr in NamedQueries verwendet werden.

alt: CUSTOM(a)

neu: FUNCTION("CUSTOM", a)

Siehe auch <https://docs.hibernate.org/orm/6.0/migration-guide/#proc-call-nativequery>

Queries ohne Select

Runtime Fehler!

```
Java

entityManager.createQuery(
    "from LowWaterWateringRule r where r.sensor.deviceId=:sensorName")
```

liefert jetzt List<LowWateringRule>, früher List<Object>.

Lösung:

```
Java

entityManager.createQuery(
    "select r from LowWaterWateringRule r where r.sensor.deviceId=:sensorName")
```

Siehe <https://docs.hibernate.org/orm/6.0/migration-guide/#query>

Keine automatischen Casts z.B. bei Vergleichen

Integer

z.B: typ == 1 wenn typ ein char(2) ist muss auf mit einem String verglichen werden. z.B: typ == "01"
Das stimmt aber eventuell nicht, wenn "1" in der DB steht

Datum

alt: current_date() - 30

neu: current_timestamp() - 30 DAY

alt: l3kaaufenthaltunterbrechung_.unterbrechvonDat + 1 = TO_DATE(:db_kaunt_von_dat, 'YYYYMMDD')

neu: CAST(l3kaaufenthaltunterbrechung_.unterbrechvonDat AS TIMESTAMP) + 1 DAY = TO_TIMESTAMP(:db_kaunt_von_dat, 'YYYYMMDD')

Updateable und Insertable

find: '@Column(name = "WERT", nullable = false, updatable = false)'

replace: '@Column(name = "WERT", nullable = false, updatable = false, insertable = false)'

Anpassung bei @FilterDef und @ParamDef

Im Zuge des Upgrades erfordert das `type`-Attribut innerhalb der `@ParamDef`-Annotation nun zwingend ein Klassen-Literal statt eines einfachen Strings. Die Unterlassung dieser Anpassung führt zu einem Konvertierungsfehler beim Kompilieren (*incompatible types*).

Vergleich der Syntax:

alt: `@ParamDef(..., type = "string")`

neu: `@ParamDef(..., type = String.class)`

Für die flächendeckende Korrektur in `*ViewBase.java` Dateien kann das Rezept `org.openrewrite.text.FindAndReplace` genutzt werden, da spezialisierte Annotation-Rezepte bei verschachtelten Strukturen teilweise unzuverlässig agieren.

Zusätzlich erzwingt Hibernate 6 die Eindeutigkeit von Filtern. Mehrfache Definitionen desselben Namens lösen nun eine `AnnotationException` aus, was eine Bereinigung redundanter Deklarationen erforderlich macht.

Evolution der UserTypes

<https://docs.hibernate.org/orm/6.0/migration-guide/#type>

Die Handhabung benutzerdefinierter Typen wurde grundlegend modernisiert. Dies ist besonders bei der Behandlung von Fixlängen-Char-Spalten (Oracle) relevant. Das bisherige `@TypeDef`-Konstrukt entfällt vollständig; stattdessen erfolgt die Referenzierung der Implementierungsklasse direkt via `@Type(value = ...)`.

Die Implementierung des `UserType`-Interface erfordert nun eine Typ-Parametrisierung (z.B. `implements UserType<String>`). Aufgrund der individuellen Logik in diesen Klassen ist eine automatisierte Migration mittels OpenRewrite hier nur schwer realisierbar. Eventuell hilft <https://docs.openrewrite.org/recipes/hibernate/typeannotationparameter> ?

Hibernate Validator und EL-Abhängigkeiten

In Testumgebungen setzt der Validator für die Message-Interpolation eine EL-Implementierung voraus. Mit dem Wechsel auf Jakarta EE hat sich hierbei die ArtifactID geändert:

alt: `org.glassfish:javax.el`

neu: `org.glassfish:jakarta.el` (empfohlene Version: 4.0.2)

Explizite Schema-Trennung in @Table

Hibernate 6.2 interpretiert Punkt-Notationen im `name`-Attribut (z.B. "SCHEMA.TABLE") fälschlicherweise als monolithischen Tabellennamen, was zu "Table not found" Fehlern führt.

Lösung: Die Definition muss strikt in die Attribute `name` und `schema` aufgeteilt werden, um korrektes SQL-Quoting zu gewährleisten.

Rückgabetypen bei Native Queries

Runtime!

Unter Verwendung des Oracle-Dialekts liefert Hibernate nun vermehrt `Long` anstelle von `BigDecimal` zurück. Um abwärtskompatibel zu bleiben und `ClassCastException`s zu vermeiden, sollte im Java-Code gegen das Interface `Number` programmiert werden.

Integer Overflow bei NUMBER(10)

Runtime!

Die implizite Zuordnung von `NUMBER(10)` zu Java-Integern führt bei großen Werten (z.B. Versicherungsnummern) zu Overflows. Abhilfe schafft ein explizites `@SqlResultSetMapping` mit Zieltyp `Long.class`.

Alternativ kann der Dialekt global erweitert werden (`TA2migOracleDialect`), um den `SqlTypeDescriptor` für diese Präzision permanent auf `BigIntJdbcType` umzulenken. Dies minimiert den manuellen Anpassungsaufwand im Transformer-Code, birgt jedoch Abhängigkeiten zur internen Hibernate-Struktur.

Bulk-Operationen und HTE-Tabellen

Bei Verwendung von Batch-Strategien (z.B. `increment_size > 1`) forciert Hibernate nun die Anlage temporärer Tabellen mit dem Präfix `HTE_`. Hibernate rät dringend dazu, diese Tabellen manuell im Datenbankschema zu hinterlegen, da automatische Inline-Strategien in künftigen Releases entfallen werden.

Vergleich mit NULL in HQL / JPQL

Die Handhabung von NULL-Werten folgt nun strikt dem SQL-Standard. Ein Vergleich via Ungleichheitsoperator liefert nicht mehr dieselben Resultate wie die explizite Prüfung auf Vorhandensein.

Beispiel:

alt: `select obj from Entity obj where obj.value != NULL` (ungültiger Ausdruck, liefert kein Ergebnis, selbst wenn der Wert ungleich NULL ist)

neu: `select obj from Entity obj where obj.value is not NULL` (korrekter Ausdruck)

Strengere Detached Prüfung

Die Validierung von Detached Entities wurde verschärft, was insbesondere bei Integrationstests zu Tage tritt. Das händische Setzen von Primärschlüsseln oder @Version-Feldern verhindert unter Umständen das Persistieren.

Lösung: Verzicht auf die manuelle Zuweisung dieser technischen Werte. Eine tiefergehende Analyse der zugrundeliegenden Logik steht noch aus.

Cast as date in @Filter

Innerhalb von Filter-Definitionen wird der Typ *date* fälschlicherweise als Variable interpretiert und mit einem automatischen Alias versehen, was zu fehlerhaftem SQL führt:

Fehler: `cast(ERSTELL_TS as date) -> cast(vo1_0.ERSTELL_TS as vo1_0.date)`

Lösung: Explizite Definition eines Alias für die Filter-Bedingung:

```
@Filter(name = ..., aliases = @SqlFragmentAlias(alias = "sa", entity = Strafanzeige.class), condition = "cast({sa}.ERSTELL_TS as date) BETWEEN ...")
```

Hibernate Java Time Handling

Bei Verwendung von @TemporalType=DATE erfolgt die Interpretation nun ausschließlich als Datum ohne Uhrzeit. Dies führt bei Oracle-Spalten vom Typ *DATE* zu Datenverlusten zur Laufzeit.

Quickfix: Umstellung auf @TemporalType=TIMESTAMP für betroffene Attribute.

Hintergrund: <https://hibernate.atlassian.net/browse/HHH-10249>